

Guida allo Sviluppo Android — Progetto Loquace

Documentazione tecnica per il passaggio di consegne

1. Introduzione allo Sviluppo Android

Android è un sistema operativo mobile basato su Linux, sviluppato da Google. Le app Android sono scritte principalmente in Kotlin (o Java) e compilate in bytecode eseguibile dalla Dalvik Virtual Machine (DVM) o dall'Android Runtime (ART). Il codice viene impacchettato in un file APK (Android Package) o AAB (Android App Bundle) per la distribuzione.

1.1 Componenti Fondamentali

- **Activity** — Rappresenta una singola schermata con interfaccia utente. Nel progetto Loquace, le principali Activity sono MainActivity (schermata principale), CallActivity (gestione chiamate) e LoquaceLoginActivity (login).
- **Fragment** — Componente UI riutilizzabile che vive all'interno di una Activity. Loquace usa decine di Fragment per le varie sezioni: ConversationsListFragment, ContactsListFragment, ActiveCallFragment, ecc.
- **ViewModel** — Componente dell'architettura MVVM che gestisce i dati dell'UI e sopravvive alle rotazioni dello schermo. Esempi: CurrentCallViewModel, XmppConversationsListViewModel, LoquaceDialerViewModel.
- **LiveData** — Oggetto osservabile che notifica i Fragment/Activity quando i dati cambiano. Usato estensivamente per aggiornare l'UI in modo reattivo.
- **Room** — Libreria di persistenza dei dati che fornisce un'astrazione su SQLite. Loquace usa Room con SQLCipher per cifrare il database locale (conversazioni XMPP, account SIP).
- **Service** — Componente che esegue operazioni in background. Loquace usa XmppConnectionService per la connessione XMPP e CoreService di Linphone per mantenere viva la connessione SIP.
- **BroadcastReceiver** — Riceve eventi di sistema o dell'app. Usato per gestire le azioni delle notifiche (rispondere/rifiutare chiamate, toggle speaker).

1.2 Architettura MVVM

Loquace adotta il pattern Model-View-ViewModel (MVVM), raccomandato da Google per le app Android moderne:

- **Model** — Dati e logica di business (repository, DAO, entità Room, classi di rete Retrofit).
- **View** — Fragment e Activity con layout XML. Osservano il ViewModel tramite LiveData.
- **ViewModel** — Gestisce lo stato dell'UI e interagisce con il Model. Non ha riferimenti diretti alla View.

1.3 Data Binding

Loquace (come Linphone) usa Android Data Binding per collegare i layout XML direttamente alle proprietà dei ViewModel. Questo significa che quando un LiveData cambia, l'UI si aggiorna automaticamente senza codice boilerplate nel Fragment. I layout che usano Data Binding iniziano con il tag `<layout>` e contengono una sezione `<data>` con le variabili. I Binding Adapter personalizzati (in `DataBindingUtils.kt`) permettono di definire attributi XML custom come `app:loquacePresence` o `app:thumbTint`.

2. Strumenti e Ambiente di Sviluppo

2.1 Android Studio

Android Studio è l'IDE ufficiale per lo sviluppo Android, basato su IntelliJ IDEA. Strumenti chiave da conoscere:

- **Logcat** — Visualizzatore di log in tempo reale. Filtrare per tag (es. LoquaceFCM, LoquaceSip) o per package (it.nems.loquacemobile) per trovare problemi rapidamente.
- **Layout Inspector** — Permette di ispezionare la gerarchia delle View a runtime, utile per trovare view nascoste o identificare ID di componenti.
- **Build > Generate Signed Bundle/APK** — Per generare la build di release firmata da caricare su Play Store.
- **Device Manager** — Per creare e gestire gli emulatori Android Virtual Device (AVD).

2.2 Gradle

Gradle è il sistema di build di Android. I file principali sono:

- **app/build.gradle.kts** — Configurazione del modulo app: applicationId (it.nems.loquacemobile), versionCode, versionName, minSdk, targetSdk, compileSdk, dipendenze.
- **loquace-integration/build.gradle.kts** — Configurazione del modulo di integrazione Loquace con le sue dipendenze specifiche (Retrofit, Room, SQLCipher, Smack, ecc.).
- **libs.versions.toml** — Catalogo centralizzato delle versioni delle dipendenze.

2.3 ADB (Android Debug Bridge)

ADB è uno strumento a riga di comando per comunicare con dispositivi Android. Comandi utili:

- **adb logcat** — Visualizza i log del dispositivo in tempo reale.
- **adb shell run-as it.nems.loquacemobile** — Accede al filesystem dell'app per debug (cancellare database, verificare file, ecc.).
- **adb install app.apk** — Installa un APK sul dispositivo.

3. Architettura del Progetto Loquace

Loquace è un'app VoIP e chat aziendale costruita su Linphone Android (branch release/6.2). Il progetto è diviso in due moduli principali:

3.1 Modulo app

Contiene il codice originale di Linphone, modificato per integrare le funzionalità Loquace. Le modifiche principali riguardano:

- **MainActivity** — Punto di ingresso principale. Gestisce la sessione Loquace, il reindirizzamento al login, la connessione XMPP e il flag `needs_relogin` per la corruzione del database.
- **CallActivity e fragment chiamate** — Gestione delle schermate di chiamata (in arrivo, in uscita, attiva). Modificati per forzare audio-only per le chiamate in arrivo, gestire il default audio (vivavoce disabilitato), e mostrare il picker contatti Loquace per nuova chiamata e trasferimento.
- **NotificationsManager** — Gestione notifiche. Modificato per nascondere il pulsante video nelle chiamate in arrivo (FreeSWITCH include sempre video nel SDP) e usare le iniziali del contatto nell'avatar della notifica.
- **DrawerMenuFragment** — Menu laterale completamente ridisegnato con sezioni Loquace: chiamate in entrata (toggle Mobile/Browser/Telefono), presenza, impostazioni, lingua, logout.
- **ContactsListFragment** — Lista contatti con tre tab: Dispositivo (contatti nativi Android), Interno (contatti PBX Loquace), Utenti (utenti Loquace). Aggiunta la mappa di presenza e la correzione della race condition tra tab.

3.2 Modulo loquace-integration

Modulo separato che contiene tutta la logica specifica di Loquace, separata dal codice Linphone per facilitare i merge futuri. Package principale: `org.linphone.loquace_integration`.

- **network/** — API Retrofit (ContactsApi, ChatsApi, CallsApi, AuthApi, ecc.), modelli di richiesta/risposta, LoquaceLogoutManager, EmergencyCallUtils.
- **storage/** — Database Room cifrato con SQLCipher (LoquaceDatabase), DAO, entità, SessionManager per la gestione del token e delle preferenze.
- **sip/** — LoquaceSipConfigurator per la configurazione dell'account SIP su Linphone Core, con gestione push notifications e token FCM.
- **xmpp/** — LoquaceXmppManager (singleton per la connessione XMPP), XmppConnectionService, XmppMessage, XmppConversation, gestione MAM (Message Archive Management) per la cronologia messaggi.
- **ui/** — LoquaceLoginActivity con schermata di login ridisegnata, `activity_login.xml`.
- **utils/** — EmergencyCallUtils (rilevamento numeri di emergenza e chiamata GSM), GroupIconUtils (icona personalizzata per i gruppi chat).

4. Scelte Implementative Chiave

4.1 Database Cifrato con SQLCipher

Il database Room locale è cifrato con SQLCipher per proteggere i dati sensibili (conversazioni, account, token) in caso di accesso fisico al dispositivo. La chiave di cifratura è generata casualmente al primo avvio e salvata in EncryptedSharedPreferences (protette dal Keystore Android). La validazione del database all'avvio previene crash da corruzione: se il database risulta invalido, viene impostato il flag `needs_relogin` in `loquace_flags` (SharedPreferences semplici) e al prossimo avvio di MainActivity viene eseguito un logout completo.

4.2 Smack per XMPP

Smack è la libreria Java/Kotlin per il protocollo XMPP usata per la chat. Funzionalità implementate:

- **MAM (Message Archive Management)** — Recupero della cronologia messaggi dal server. Attenzione: i timestamp dei messaggi MAM si trovano nel wrapper `forwarded.delayInformation`, non nel messaggio interno. Usare `result.mamResultExtensions` invece di `result.messages`.
- **MUC (Multi-User Chat)** — Gruppi chat. L'utente deve unirsi alla stanza (`joinRoom`) prima di ricevere messaggi.
- **Connessione** — La connessione XMPP viene stabilita quando l'app è in foreground (`onStart`) e disconnessa quando va in background (`onStop`). Le notifiche push FCM gestiscono i messaggi in background.

4.3 Retrofit per le API REST

Retrofit è la libreria usata per le chiamate HTTP alle API Loquace. Il dominio del server viene salvato in `SessionManager` e usato per costruire l'URL base in `RetrofitClient`. Le API principali sono: `ContactsApi`, `ChatsApi` (gruppi XMPP), `CallsApi` (avvio chiamate), `AuthApi` (login), `PresenceApi`, `SettingsApi`. Prima di ogni chiamata, l'app intercetta il numero/contatto tramite `POST /api/v2/calls` per permettere al backend di reindirizzare o modificare il numero chiamato.

4.4 Linphone SDK per SIP/VoIP

Linphone è un progetto open-source VoIP su cui è basato Loquace. Il codice sorgente è il branch `release/6.2` di `linphone-android`. Punti critici da conoscere:

- **FreeSWITCH e video SDP** — Il backend FreeSWITCH include sempre `m=video` nel SDP anche per chiamate audio. Questo causa problemi: notifica in arrivo mostra pulsante video, vivavoce si attiva automaticamente. Le correzioni sono in `NotificationsManager` (`isVideo=false` per incoming), `CurrentCallViewModel` (`videoExplicitlyEnabled` flag) e `TelecomCallControlCallback` (metodo `video routing` commentato).
- **Push e registrazione SIP** — Dopo un data wipe, il token FCM arriva prima che il core Linphone sia inizializzato. La soluzione è salvare il token in `loquace_flags` (`pending_fcm_token`) e applicarlo in `LoquaceSipConfigurator.configure()` prima di `addAccount()`. Inoltre, all'arrivo di un push con `call-id`, viene forzato un `refreshRegister()` per garantire che FreeSWITCH abbia l'indirizzo di contatto aggiornato prima di inviare l'INVITE SIP.
- **Keep Alive Service** — Il servizio keep alive di Linphone mantiene la connessione SIP attiva in background. Viene abilitato in `PermissionsFragment` dopo che il permesso `POST_NOTIFICATIONS` è concesso, per evitare crash su Android 13+.

4.5 Firebase Cloud Messaging (FCM)

`LoquaceFirebaseMessagingService` estende `FirebaseMessaging` di Linphone e instrada i push in tre categorie:

- **call-id presente** → Push SIP: `refreshRegister()` + `super.onMessageReceived()` per far gestire la chiamata a Linphone.
- **subject=chat** → Notifica messaggio chat XMPP: mostra notifica con mittente e testo.

- **altri casi** → Push SIP generico: `super.onMessageReceived()`.

5. Flusso di Login e Sessione

Il flusso di autenticazione è completamente gestito da Loquace, sostituendo il sistema di login nativo di Linphone:

- **1. LoquaceLoginActivity** — L'utente inserisce dominio, username e password. L'app chiama le API REST Loquace per autenticarsi e ricevere il token.
- **2. SessionManager** — Salva il token, il dominio e altri dati in EncryptedSharedPreferences. Il token è il punto di riferimento principale per verificare se l'utente è loggato (isLoggedIn()).
- **3. LoquaceSipConfigurator** — Configura l'account SIP su Linphone Core con le credenziali ricevute dal backend.
- **4. PermissionsFragment** — Dopo il login, vengono richiesti i permessi necessari (RECORD_AUDIO, CAMERA, READ_CONTACTS, POST_NOTIFICATIONS, CALL_PHONE).
- **5. MainActivity** — Controlla SessionManager.isLoggedIn() come fonte di verità principale, prima di verificare lo stato di Linphone Core.

5.1 Logout

LoquaceLogoutManager.logout() esegue il logout in ordine: disconnessione XMPP → rimozione account SIP → cancellazione file config Linphone → cancellazione avatar → chiusura e cancellazione database → pulizia sessione. È importante l'ordine: il database va cancellato dopo che XMPP si è disconnesso, altrimenti XMPP tenta di accedere al database che non esiste più.

5.2 Prevenzione Accesso Non Autorizzato

- **android:allowBackup="false"** — Impedisce ad Android di fare backup/ripristino del database cifrato e dei dati di sessione al reinstall.
- **android:noHistory="true" su LoquaceLoginActivity** — Rimuove il login dal back stack dopo l'accesso.
- **OnBackPressedCallback in LoquaceLoginActivity** — Chiama moveTaskToBack(true) per mandare l'app in background invece di navigare verso lo skeleton Linphone.

6. Gestione Chiamate SIP

6.1 Avvio Chiamata

Prima di avviare qualsiasi chiamata, l'app intercetta il numero tramite POST `/api/v2/calls`. Il payload varia in base al contesto:

- **Tastiera** → `{"contact":{"number":"215"}}`
- **Contatto SIP (tab Utenti)** → `{"contact":{"id":"contactId","number":"215"}}`
- **Contatto dispositivo (tab Dispositivo)** → `{"contact":{"name":"Rossi Mario","number":"3481234567"}}`

La risposta contiene `contact.number` che è il numero effettivo da chiamare (il backend può modificarlo). Con quel numero si costruisce l'indirizzo SIP tramite `core.interpretUrl()` e si avvia la chiamata con `coreContext.startAudioCall()`.

6.2 Numeri di Emergenza

Se il numero è rilevato come emergenza tramite `TelephonyManager.isEmergencyNumber()` (API 29+), l'app bypassa completamente il VoIP e avvia una chiamata GSM tradizionale tramite `EmergencyCallUtils.placeGsmCall()`. L'intent viene indirizzato esplicitamente al dialer di sistema tramite `TelecomManager.defaultDialerPackage` per evitare il dialog di selezione app.

6.3 Schermata di Chiamata

`CurrentCallViewModel` gestisce tutta la logica della schermata di chiamata attiva. Modifiche principali rispetto a Linphone originale:

- **Recupero contatto Loquace** — In `configureCall()`, viene chiamata l'API `/api/v2/contacts?phones=sipNumber` per ottenere nome completo e avatar del chiamante.
- **Gestione audio** — `speakerExplicitlyEnabled` e `videoExplicitlyEnabled` sono flag che impediscono a Linphone di attivare automaticamente vivavoce e video per le chiamate in arrivo (dovuto al video nel SDP di FreeSWITCH).
- **Sfondo bianco** — I layout `call_active_fragment.xml`, `call_incoming_fragment.xml` e `call_outgoing_fragment.xml` sono stati modificati con sfondo bianco invece del nero originale.

7. Chat XMPP

7.1 Architettura Chat

La chat è completamente separata da Linphone e implementata nel modulo loquace-integration con Smack. Le conversazioni hanno tre tab nella schermata chat:

- **Conversazioni** — Conversazioni aperte, filtrate client-side da `LoquaceXmppManager.conversations` `StateFlow`.
- **Contatti** — Contatti con XMPP abilitato, caricati tramite API con paginazione e ricerca server-side (parametro `term=`).
- **Gruppi** — Gruppi MUC, caricati via API e filtrati client-side.

7.2 Timestamp dei Messaggi

I timestamp vanno gestiti con attenzione: Linphone (`TimestampUtils`) si aspetta secondi, mentre `System.currentTimeMillis()` restituisce millisecondi. La conversione viene fatta in `XmppConversationModel.formatDateTime()`. Per i messaggi MAM, il timestamp corretto si trova in `forwarded.delayInformation?.stamp?.time`, non nel messaggio interno.

7.3 Picker Partecipanti

`ParticipantPickerBottomSheet` è usato per selezionare partecipanti alla creazione di gruppi e per aggiungere membri. Mostra solo contatti Loquace (con XMPP abilitato) poiché solo questi supportano la chat. `LoquaceCallContactPickerBottomSheet` è invece usato per nuova chiamata e trasferimento durante una chiamata attiva, e include anche i contatti del dispositivo.

8. Pubblicazione su Google Play

8.1 Firma dell'App

Per pubblicare su Play Store, l'app deve essere firmata con una chiave privata (keystore). Il keystore va conservato in modo sicuro — se perso, non si può aggiornare l'app sullo Store. In Android Studio: Build > Generate Signed Bundle/APK > Android App Bundle > selezionare il keystore > completare il processo.

8.2 Versioning

- **versionCode** — Numero intero incrementale. Google Play richiede che ogni upload abbia un versionCode maggiore del precedente.
- **versionName** — Stringa leggibile (es. "1.0.0") mostrata agli utenti.

8.3 google-services.json

Il file google-services.json contiene la configurazione Firebase (FCM) ed è escluso dal version control (.gitignore). Va ottenuto dalla Firebase Console e copiato in app/ prima di compilare. Senza questo file le notifiche push non funzioneranno.

9. Checklist Pre-Rilascio

9.1 Prima di ogni Release

- **Incrementare versionCode e versionName** — In app/build.gradle.kts.
- **Verificare google-services.json** — Deve essere presente in app/ e aggiornato.
- **Test su dispositivo fisico** — Testare il flusso completo: login, chiamata in arrivo da background, chat, logout.
- **Verificare le traduzioni** — Controllare che tutte le nuove stringhe siano presenti in res/values-it/strings.xml (attenzione: file_provider_loquace NON va tradotto).

9.2 Merge da Linphone Upstream

Quando si fa merge dal branch release/6.2 di Linphone, verificare tutte le modifiche documentate in CHANGES.md. I file più a rischio di conflitto sono:

- **AndroidManifest.xml** — Molte modifiche Loquace (allowBackup, screenOrientation, FileProvider, servizi, permessi).
- **MainActivity.kt** — Flusso di login, onResume, onStart, onStop.
- **CurrentCallViewModel.kt** — Gestione audio, video, recupero contatto Loquace.
- **NotificationsManager.kt** — Pulsante video notifica, icona chiamante.
- **res/values/strings.xml** — Tutte le stringhe Loquace aggiunte in fondo al file.

10. Note Finali e Risorse

10.1 Documentazione di Riferimento

- **CHANGES.md** — Documento che elenca tutte le modifiche apportate al codice Linphone originale. Da consultare ad ogni merge upstream e da aggiornare ad ogni modifica.
- **Android Developers** — <https://developer.android.com> — Documentazione ufficiale Android.
- **Linphone Android** — <https://github.com/BelledonneCommunications/linphone-android> — Repository originale.
- **Smack XMPP** — <https://github.com/igniterealtime/Smack> — Documentazione libreria XMPP.

10.2 Contesto della Conversazione con Claude

Tutta la storia di sviluppo di Loquace è documentata nelle conversazioni con Claude (Anthropic). Queste conversazioni possono essere esportate e usate come contesto con qualsiasi LLM per continuare il lavoro. I punti chiave da fornire come contesto a un nuovo LLM sono:

- **Base del progetto** — linphone-android release/6.2, modificato con modulo loquace-integration (package org.linphone.loquace_integration, applicationId it.nems.loquacemobile).
- **Stack tecnologico** — Kotlin, MVVM, Retrofit, Room+SQLCipher, EncryptedSharedPreferences, Firebase FCM, Smack XMPP, Linphone SDK.
- **CHANGES.md** — Fornire sempre il file CHANGES.md aggiornato come contesto: contiene tutte le modifiche al codice Linphone e le decisioni architetturali.
- **Problemi noti** — FreeSWITCH include sempre video nel SDP (workaround implementati), G.729 non disponibile nell'SDK open-source (richiede build custom con bcg729)

In bocca al lupo con il progetto!